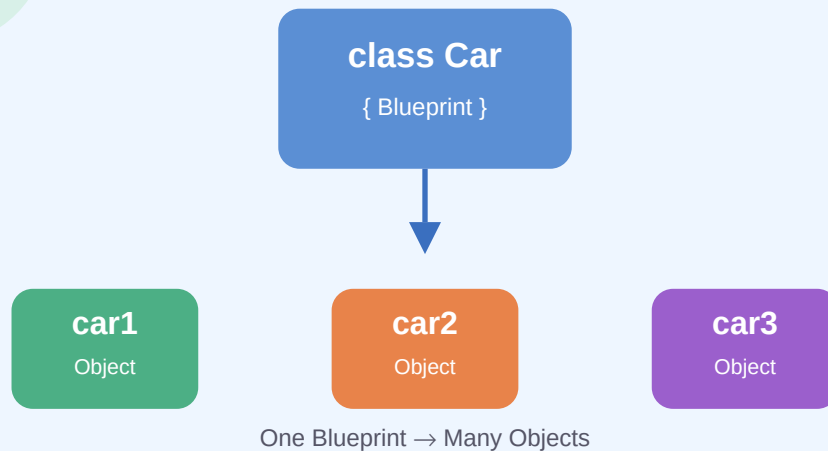


# Objects & Classes in Java

## Unit 2 • Part 1



### Blueprint • Object Factory • User-Defined Types

Message Passing • Encapsulation • State & Behavior

## ■ What is an Object?

An **object** is a real-world entity that has two things:

1. **State** – the data it holds (e.g., color, speed)
2. **Behavior** – the actions it can perform (e.g., start, brake)

## ■ Real-World Examples of Objects

| Object      | State (Attributes)   | Behavior (Methods)         |
|-------------|----------------------|----------------------------|
| ■ Car       | Color, Model, Speed  | start(), brake(), horn()   |
| ■ Dog       | Name, Breed, Age     | bark(), eat(), run()       |
| ■■■ Student | Name, Roll No, Marks | study(), write(), answer() |

## ■ What is a Class?

A **class** is a **blueprint** or **template** for creating objects. It defines what attributes and methods an object will have. A class does not occupy memory — memory is allocated only when an object is created.

## ■ Class as a Blueprint – Analogy

### ■ Architect's Blueprint → House

Just as one blueprint can be used to build many houses, one **class** can be used to create many **objects**.

### ■ Biscuit Mould → Biscuits

The mould is the class. Every biscuit made from it is an object — same shape, different position!

## ■■ Java Syntax – Defining a Class

```
class Car {  
    String color; // attribute  
    String model; // attribute  
    int speed; // attribute  
}
```

## ■ Class as an Object Factory

A class works like a **factory**. You write the class **once**, and then you can create as many objects as you need using the **new** keyword.

```
Car car1 = new Car(); // Object 1
Car car2 = new Car(); // Object 2
Car car3 = new Car(); // Object 3

car1.color = "Red";
car2.color = "Blue";
car3.color = "Green";
```

■ All three objects are created from the same class **Car**, but each has its own independent data.

## ■ Class as a User-Defined Data Type

| Primitive Data Type | User-Defined Data Type                     |
|---------------------|--------------------------------------------|
| <code>int x;</code> | <code>Car myCar;</code>                    |
| Defined by Java     | Defined by YOU (programmer)                |
| Holds simple values | Holds complex objects with data + behavior |

## ■ Complete Java Example

```
class Dog {
    String name;
    String breed;
    int age;
}

class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.name = "Bruno";
        d1.breed = "Labrador";
        d1.age = 3;

        Dog d2 = new Dog();
        d2.name = "Moti";
        d2.breed = "Pomeranian";
        d2.age = 2;

        System.out.println(d1.name + " is a " + d1.breed);
        System.out.println(d2.name + " is a " + d2.breed);
    }
}
```

**Output :**

```
Bruno is a Labrador
Moti is a Pomeranian
```

## ■ Three Ways to Think About a Class

| ■ Blueprint                                             | ■ Factory                                               | ■■ Data Type                                               |
|---------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------|
| Defines the structure of objects (attributes & methods) | Produces multiple objects — each independent, each real | A new type defined by the programmer with its own behavior |

## ■ Quick Revision Summary

| Term             | Meaning                                            |
|------------------|----------------------------------------------------|
| Class            | Blueprint / Template / User-Defined Data Type      |
| Object           | An instance created from a class using 'new'       |
| Attribute        | Variable inside a class that stores object's state |
| new keyword      | Allocates memory and creates an object             |
| Dot operator (.) | Accesses attributes or methods of an object        |
| State            | Current data/values stored in an object            |
| Behavior         | Actions an object can perform (via methods)        |

## ■ Memory Trick – C.O.B.

**C** → **Class** is the **Cook book recipe** ■  
**O** → **Object** is the actual **dish made** ■■  
**B** → **Blueprint** stays same; dishes differ ■

Remember: **One Recipe** → **Many Dishes**  
 Just like: **One Class** → **Many Objects**

## ⇒ ■ Practice Questions

1. What is a class in Java? How is it different from an object?
2. Explain class as a user-defined data type with an example.

3. Write a class **Student** with attributes: name, rollNo, marks. Create two objects and assign values.
4. What does the **new** keyword do in Java?
5. Why is a class called a blueprint? Give a real-life analogy.
6. Can two objects of the same class have different data? Explain with code.
7. Identify the class, object, attributes, and methods in: *A Bank Account has an account number, balance, and can deposit or withdraw money.*
8. What is the dot (.) operator used for in Java?

### ■ ■ Important Tips for Exams

- ✓ A class does **not** occupy memory — memory is allocated only when an object is created.
- ✓ Each object has its own copy of the instance variables (attributes).
- ✓ The **new** keyword is mandatory to create an object in Java.
- ✓ Use the **dot (.) operator** to access attributes and methods of an object.
- ✓ A class can be defined anywhere in the program — order doesn't matter in Java.
- ✓ Class name should start with a **Capital letter** (convention: PascalCase).